

Введение в CMake

1. Что, зачем и для чего

CMake – кроссплатформенная система автоматизации сборки, которая поддерживается большинством современных систем разработки (IDE).

У многих IDE есть свои системы автоматизации сборки (например, Visual Studio, Qt Creator), но использование CMake избавит вас от зависимости от среды разработки. Для полноценной сборки проекта не всегда достаточно исходных файлов с кодом, в некоторых случаях критически важно использование конкретной версии стандарта, использование конкретных флагов для компилятора. Эта информация хранится в других файлах проекта, в формате, который требуется конкретной системе сборки. Использование одной и той же системы, поддерживаемой разными IDE, позволяет без проблем дома на Visual Studio дорабатывать код, который вы начали писать на ЛР в НОЦ ИБ в CLion.

2. Как это использовать

В CMake вся информация, необходимая для сборки, хранится в конфигурационном файле, который называется CMakeLists.txt.

Например, когда вы создаёте в CLion новый проект, в каталоге проекта создаётся файл с исходным кодом, файл CMakeLists.txt и каталог для отладочной (debug) сборки (build).

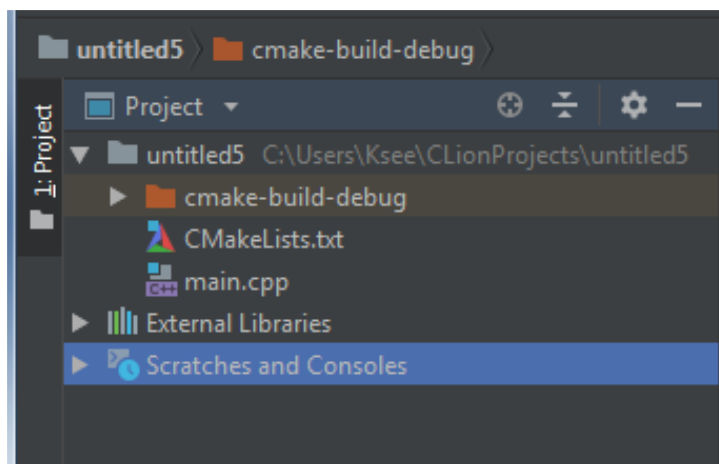


Рис. 1. Файлы проекта

Рассмотрим содержимое файла CMakeLists.txt, который сгенерировал CLion по умолчанию (пример 1).

Пример 1. Содержимое CMakeLists.txt.

```
1 cmake_minimum_required(VERSION 3.12)
2
3 project(untitled5)
4
5 set(CMAKE_CXX_STANDARD 17)
6
7 add_executable(untitled5 main.cpp)
```

Итак, первая строка проверяет минимально необходимую для сборки проекта версию CMake. При создании проекта сюда вписывается версия CMake, используемая средой разработки. Строка 3 задаёт название проекта, с которым связан текущий CMakeLists. Строка 5 определяет используемый стандарт C++ (в примере это 17 – C++17). Строка 7 определяет цель (target) – исполняемый файл или библиотека, который будет построен в результате сборки проекта. В данном случае цель – untitled5, которая собирается из main.cpp. Первым аргументом задаётся имя исполняемого файла, а далее через пробелы перечисляются файлы заголовков и файлы с исходным кодом, которые должны быть скомпилированы и слинкованы для получения исполняемого файла.

При создании класса через IDE (рис. 2) по умолчанию добавляет класс в «цели». После добавления класса CMakeLists будет выглядеть как в примере 2.

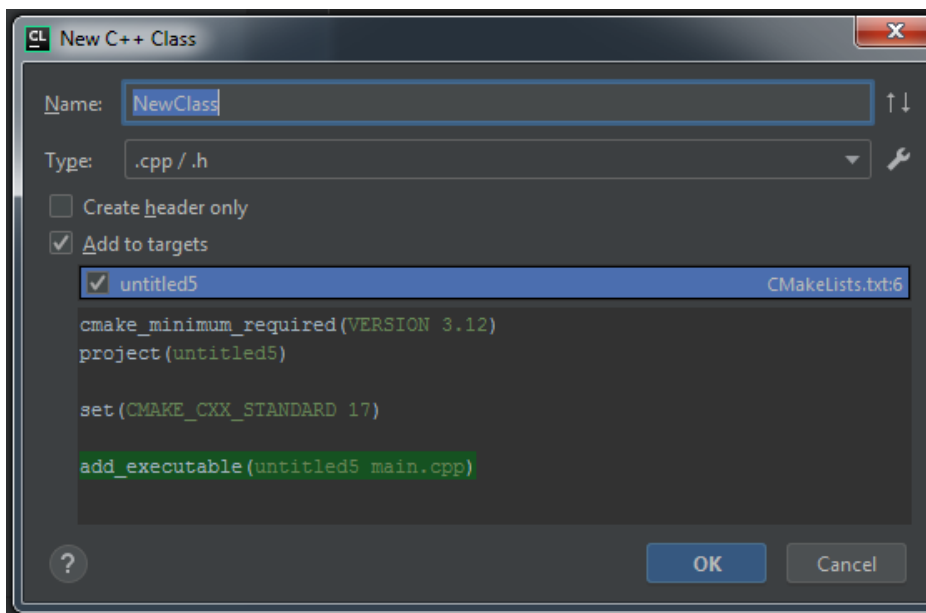


Рис. 2. Создание класса через интерфейс CLion

Пример 2. Содержимое CMakeLists.txt после добавления класса

```
1 cmake_minimum_required(VERSION 3.12)
2
3 project(untitled5)
4
5 set(CMAKE_CXX_STANDARD 17)
6
7 add_executable(untitled5 main.cpp NewClass.cpp NewClass.h)
```

Если вы добавляете файлы без добавления в цели и добавляете их в цели позже, то обращайтесь внимание на расположение файла. Если вы добавили файлы не в тот же каталог, где находится CMakeLists.txt, должно быть указано не только название файла, но и путь к нему.

Можно изменить имя цели. Для того, чтобы принять изменения, нужно нажать «Reload changes» в появившейся плашке (рис. 3) или «Reload CMake Project» в меню по ПКМ (рис. 4). Также можно разрешить автоматическую загрузку изменений (рис. 3).

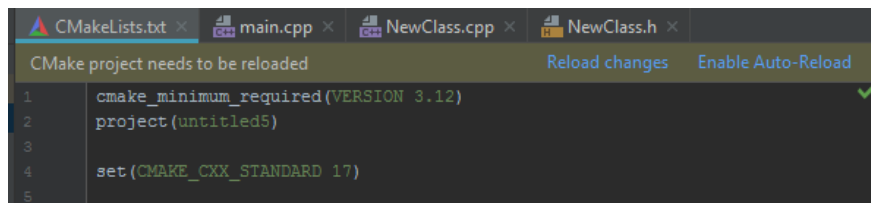


Рис. 3. Плашка сверху, появившаяся после изменения кода в CMakeLists

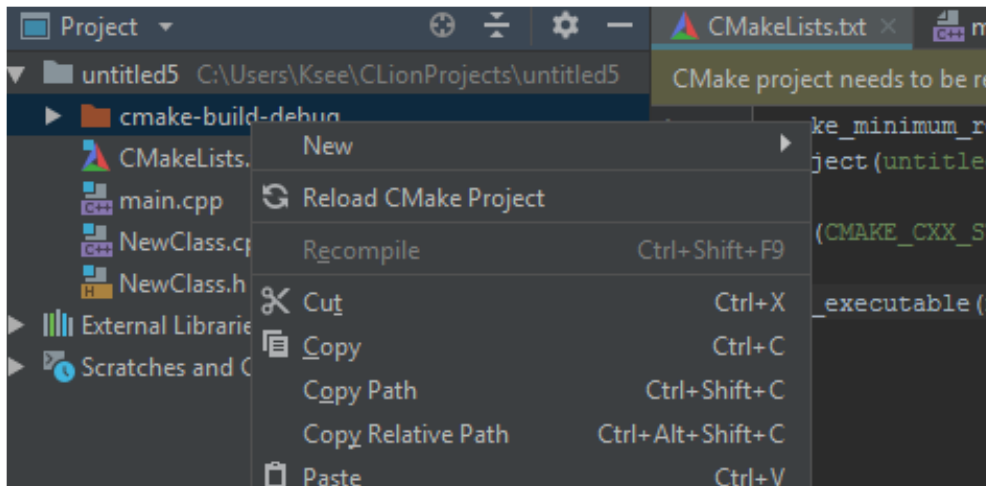


Рис. 4. Принятие изменений через меню по ПКМ.

Можно указать несколько целей. То, какая будет собрана, выбирается через меню (рис. 5).

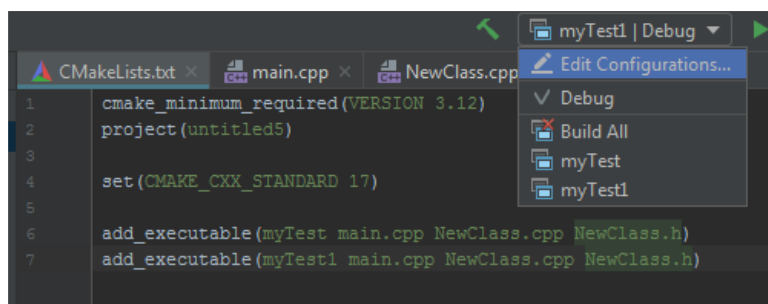


Рис. 5. Выбор цели для построения решения.

Вы, вероятно, заметили, что код в CMakeLists.txt – это не код на C++. Там используется свой язык с другим синтаксисом. В частности, комментарии в коде CMake должны начинаться с решётки (#).

3. Что ещё можно делать в CMakeLists

Вообще говоря, очень много всего. В нём есть логические операторы (NOT, AND, OR), условный оператор if() с ветвями else() и elseif() и завершающим оператором endif(), операторы сравнения (LESS, GREATER, EQUAL, LESS_EQUAL, MATCHES для использования регулярных выражений и т.д.), циклы while()-endwhile(), foreach()-endforeach() и даже функции и макросы. Вопрос в том, какую цель вы перед собой ставите.

Например, конфигурация с проверкой компилятора будет выглядеть как-то так (пример 3). Команда set позволяет присвоить переменной, переданной первым аргументом, значение, переданное

вторым аргументом. Строки 9 и 14 присваивают последовательность флагов переменной EXTRA_FLAGS, которая потом конкатенируется с имеющимися флагами переменной CMAKE_CXX_FLAGS и записывается обратно в CMAKE_CXX_FLAGS (строки 11, 15). Команда message позволяет вывести на экран сообщение. Через \${} осуществляется получение значения переменной. С помощью условного оператора значение переменной, отвечающее за используемый компилятор (CMAKE_CXX_COMPILER_ID), проверяется на значение “GNU” для компилятора g++ или “MSVC” для компилятора Visual Studio.

Пример 3. Выставление флагов для разных компиляторов

```
1 cmake_minimum_required(VERSION 3.12)
2
3 project(untitled5)
4
5 set(CMAKE_CXX_STANDARD 17)
6
7 if (CMAKE_CXX_COMPILER_ID STREQUAL "GNU")
8     message("Compiler is GNU")
9     set(EXTRA_FLAGS "-pedantic-errors -Wall -Wextra -Weffc++ -Wsign-conversion -
10 D_GLIBCXX_DEBUG -D_GLIBCXX_DEBUG_PEDANTIC")
11     set(CMAKE_CXX_FLAGS "${CMAKE_CXX_FLAGS} ${EXTRA_FLAGS}")
12 elseif(CMAKE_CXX_COMPILER_ID STREQUAL "MSVC")
13     message("Compiler is MSVC")
14     set(EXTRA_FLAGS "/W4")
15     set(CMAKE_CXX_FLAGS "${CMAKE_CXX_FLAGS} ${EXTRA_FLAGS}")
16 endif()
17
18 add_executable(untitled6 main.cpp)
```

Также можно определить используемое окружение ([видео](#)).

Флаги компилятора влияют на то, как компилируется проект, какие требования должны выполняться (например, строгое соответствие стандарту, [видео](#)). Про флаги компилятора MSVC можно почитать [здесь](#). Про флаги gcc – [здесь](#). Флаги могут [подтягиваться из стандартных конфигураций](#).

4. Как работать с CMake в Visual Studio

Если при установке VS вы не устанавливали компоненты из пункта «Разработка для Linux на C++»:

1. Запустите программу «Visual Studio Installer» (можно найти через поиск на вашем компьютере)
2. Около вашей версии VS выбираете «Дополнительно» -> «Изменить» (рис. 6).
3. Находите там «Разработка для Linux на C++», проверяете, что стоит галочка напротив «средства CMake» и устанавливаете (рис. 7).

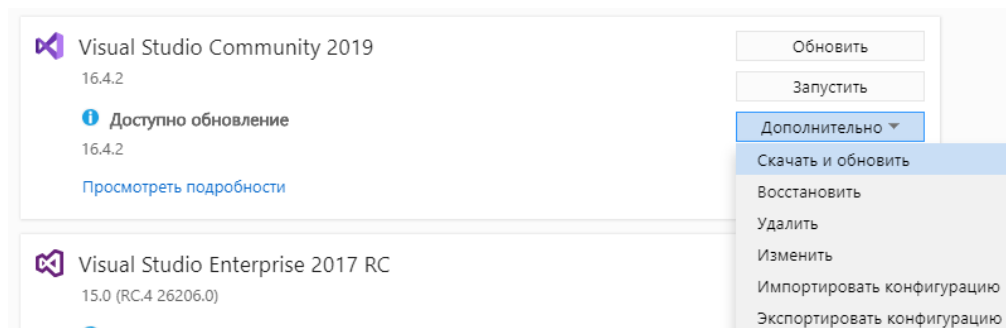


Рис. 6. Выбор в Visual Studio Installer

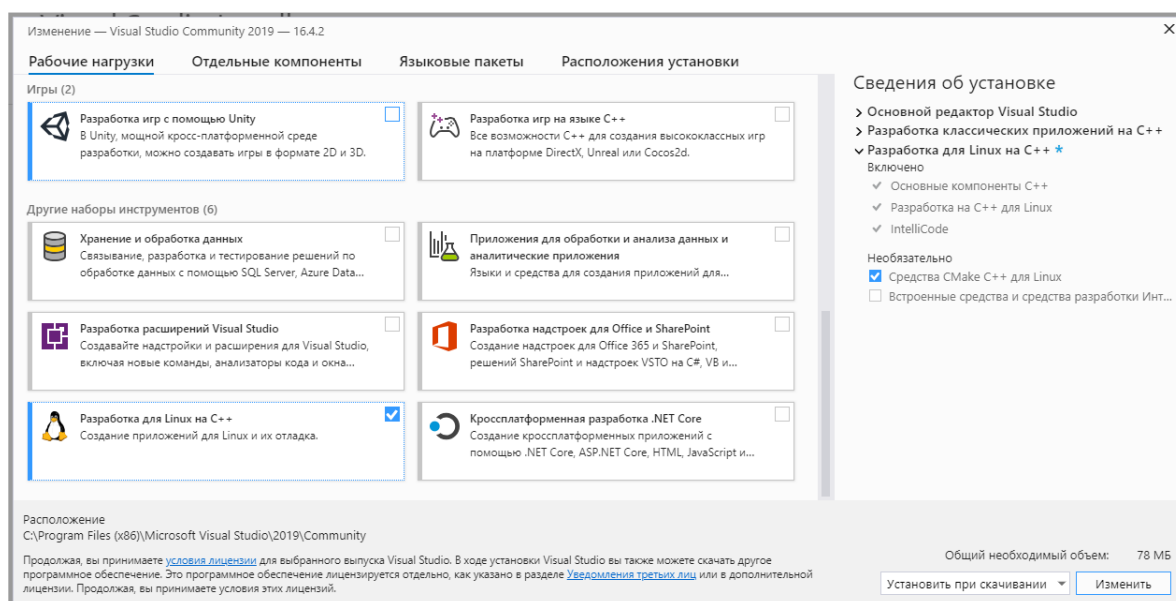


Рис. 7. Нужный компонент

Теперь при создании проекта доступен пункт «Проект CMake» (рис. 8).

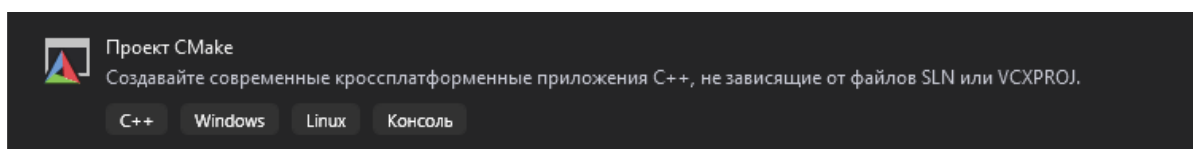


Рис. 8. Новый тип проекта

Если при создании проекта вы получаете структуру с рис. 9 (дополнительный каталог, 2 файла CMakeLists.txt), то при создании проекта у вас не стояла галочка «Поместить решение и проект в одном каталоге» (рис. 11).

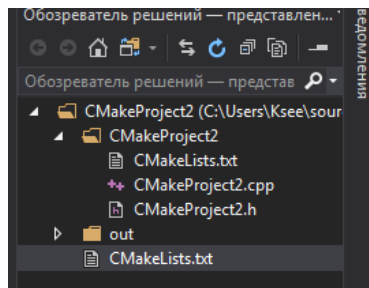


Рис. 9. Обозреватель решений, когда решение находится в отдельном каталоге

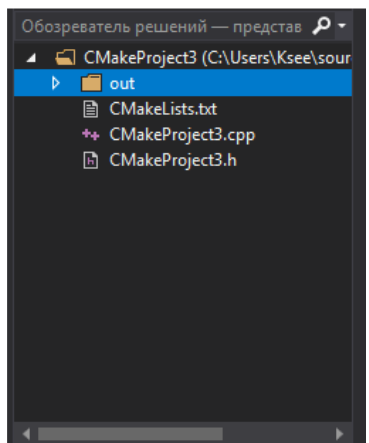


Рис. 10. Проект решением в том же каталоге

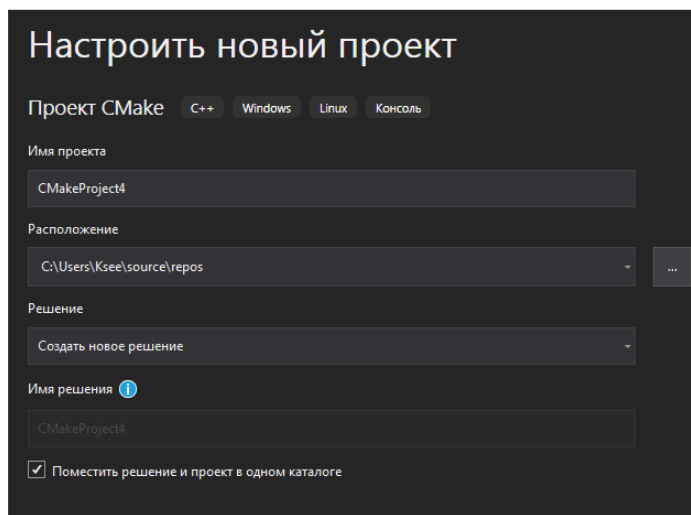


Рис. 11. Настройки проекта при создании